MSc Computer Science — Distributed Computing

Lab 04

RPC and Web Services

net/rpc | gRPC | REST | Take-Home Assignment

Item	Detail
What you build	Same key-value store using 3 protocols: net/rpc, gRPC, REST
Where you run it	Your own laptop — Docker Desktop required
Estimated time	3–4 hours
Prior lab code	Not required — fully independent
Submission	complete Moodle quiz
Questions	Post to Moodle forum

Background — Three Ways to Call a Remote Function

All three protocols in this lab solve the same problem: how does one program call a function on another program running on a different machine? They just solve it in different ways with different trade-offs.

Protocol	How it works	Language support	Used for
net/rpc	Go's standard library. Sends Go structs over TCP using gob encoding. Simple but Go-only.	Go only	Internal Go microservices
gRPC	Google's framework. Defines service in .proto file, generates client/server code in any language. Uses HTTP/2 and Protocol Buffers.	Any language	Industry APIs, microservices, Google/Netflix/Uber
REST	Standard HTTP verbs (GET, PUT, DELETE) with JSON. Any HTTP client works — browsers, curl, Python.	Any language or browser	Public APIs, web services, mobile backends

Why Learn All Three?

net/rpc: you already used this in Labs 02 and 03 — now you will understand HOW it works. gRPC: made by Google (same as Go), industry standard for microservices. The .proto file is the same concept as Java RMI's IDL. REST: the dominant style for public APIs — you can test it with a browser.

All three will be backed by the SAME shared store — put a key via REST, retrieve it via gRPC. This shows they are all just different ways to reach the same data.

Setup — Get Your Environment Running

> ⚠ **Do This First**
> Complete all setup steps before touching any code. The Docker build takes a few minutes and downloads dependencies — do it before starting the tasks.

Step 1 — Install Docker Desktop

Your OS	Download link	Notes
Windows 10/11	https://docs.docker.com/desktop/install/windows-install/	Requires WSL2 — installer guides you
macOS (Intel)	https://docs.docker.com/desktop/install/mac-install/	Intel chip version
macOS (Apple M1/M2/M3)	https://docs.docker.com/desktop/install/mac-install/	Apple Silicon version
Linux	https://docs.docker.com/desktop/install/linux-install/	Or Docker Engine + Compose plugin

Open Docker Desktop and wait for green running status. Then verify:

```
docker --version
docker-compose --version
```

Step 2 — Download Lab 04 Files

```
# Download lab04-complete.zip from Moodle and unzip
unzip lab04-complete.zip
cd lab04-complete

# Verify contents:
ls
# Should show: student/  solution/  docker/  docs/  README.md
```

Step 3 — Build the Docker Image

This step compiles your Go code AND runs protoc to generate the gRPC code automatically. It needs internet access to download Go dependencies.

```
docker build -t lab04 -f docker/Dockerfile .

# Takes 3-5 minutes on first run
# Downloads: Go compiler, protoc, gRPC libraries
# Expected last line: Successfully tagged lab04:latest

# If build fails with a syntax error in your code:
# Fix the error then run again
```

Step 4 — Start Server and Client Containers

```
docker-compose -f docker/docker-compose.yml up -d

# Verify both are running:
docker ps | grep lab04
# Should show: lab04-server  lab04-client
```

Step 5 — Verify All 3 Servers Are Running

```
# Check server logs
docker logs lab04-server
# Expected output:
# [net/rpc] Server listening on port 7000
# [gRPC]    Server listening on port 7001
# [REST]    Server listening on port 8080

# Quick test (before implementing anything — these will fail, that is correct)
docker exec lab04-client /lab04/client/client_bin rpc put test ok
# Expected: not implemented yet
```

> ☑ **Setup Complete**
> The server container runs all 3 servers. The client container is where you run tests. All source files are inside the containers at /lab04/server/ and /lab04/client/ Use docker exec -it lab04-server bash to enter the server container.

Container Reference

Container	Role	Enter with	Source files at
lab04-server	Runs all 3 servers	docker exec -it lab04-server bash	/lab04/server/
lab04-client	Run tests and benchmark	docker exec -it lab04-client bash	/lab04/client/

How to Edit, Recompile and Restart

> ⚠ **Must Recompile After Every Edit**
> Editing a file does NOT automatically update the running binary. You must rebuild after every change.

```
# Enter server container
docker exec -it lab04-server bash
cd /lab04/server

# Edit a file
nano handler.go

# Recompile
go build -o server_bin .

# Restart server
pkill server_bin
./server_bin &
```

```
# Same process for client
docker exec -it lab04-client bash
cd /lab04/client
nano rpc_client.go
go build -o client_bin .
```

CLI Reference — All Commands

```
# Run from inside lab04-client container
cd /lab04/client

# Individual protocol tests
./client_bin rpc  put <key> <value>
./client_bin grpc get <key>
./client_bin rest delete <key>
./client_bin rest list

# Test same operation with ALL 3 protocols at once
./client_bin all put city London
./client_bin all get city

# Benchmark — compare performance of all 3
./client_bin bench 100

# Test REST directly with curl (from your LAPTOP terminal, not container)
curl -X PUT http://localhost:8080/keys/city \
     -H 'Content-Type: application/json' \
     -d '{"value":"London"}'
curl http://localhost:8080/keys/city
curl http://localhost:8080/keys
curl -X DELETE http://localhost:8080/keys/city
```

Understanding the Proto File (Task 5 — Read Only)

Before implementing gRPC, read the proto file and the generated code to understand what protoc created for you.

The Service Contract — kvstore.proto

This file is in /lab04/server/proto/kvstore.proto — it defines the gRPC service interface:

```
syntax = "proto3";
package kvstore;

service KeyValueStore {
  rpc Put(PutRequest)       returns (PutResponse);
  rpc Get(GetRequest)       returns (GetResponse);
  rpc Delete(DeleteRequest) returns (DeleteResponse);
  rpc List(ListRequest)     returns (ListResponse);
}

message PutRequest    { string key = 1; string value = 2; }
message PutResponse   { bool success = 1; }
message GetRequest    { string key = 1; }
message GetResponse   { string value = 1; bool found = 2; }
message DeleteRequest { string key = 1; }
message DeleteResponse{ bool deleted = 1; }
message ListRequest   {}
message ListResponse  { repeated string keys = 1; }
```

What protoc Generated

During docker build, protoc read kvstore.proto and generated two Go files in /lab04/server/proto/:

Generated file	What it contains	You use it for
kvstore.pb.go	Go structs for every message (PutRequest, GetResponse etc.) with serialisation code	Import as pb "lab04server/proto" then use pb.PutRequest{...}
kvstore_grpc.pb.go	KeyValueStoreServer interface you must implement. KeyValueStoreClient stub for making calls.	Embed pb.UnimplementedKeyValueStoreServer in your server struct

```
# Read the generated files inside the container
docker exec -it lab04-server bash
cat /lab04/server/proto/kvstore_grpc.pb.go

# Look for the server interface — it looks like:
# type KeyValueStoreServer interface {
#     Put(context.Context, *PutRequest) (*PutResponse, error)
#     Get(context.Context, *GetRequest) (*GetResponse, error)
#     Delete(context.Context, *DeleteRequest) (*DeleteResponse, error)
#     List(context.Context, *ListRequest) (*ListResponse, error)
# }

# This is what you implement in grpc_server.go (Task 6)
```

gRPC vs Java RMI

This .proto file serves the same purpose as Java's RMI interface definition. In Java RMI you write a Java interface; in gRPC you write a .proto file. Both define the service contract that both client and server must follow. The difference: gRPC generates code for ANY language from ONE .proto file.

Part 1 — net/rpc (Tasks 1–4)

Open the server and client source files:

```
docker exec -it lab04-server bash
cd /lab04/server
nano types.go    # Task 1
nano handler.go  # Task 2
nano main.go     # Task 3

docker exec -it lab04-client bash
cd /lab04/client
nano rpc_client.go  # Task 4
```

net/rpc Tasks

Task 1 — server/types.go
`8 structs` [Easy]
Define PutArgs/Reply, GetArgs/Reply, DeleteArgs/Reply, ListArgs/Reply — all fields must be EXPORTED (capital letter)

Task 2 — server/handler.go
`Put, Get, Delete, List` [Easy]
Each method calls the correct store function and fills in the reply struct

Task 3 — server/main.go
`startRPCServer()` [Easy]
Register KVHandler, listen on TCP port 7000, call rpc.Accept(ln)

Task 4 — client/rpc_client.go
`Connect, Put, Get, Delete, List, Close` [Medium]
Dial the server, call each method using client.Call("KVHandler.Method", args, &reply)

How net/rpc Works — Key Points

Concept	Detail
Registration	rpc.Register(handler) — makes methods callable remotely
Method naming	client.Call("KVHandler.Put", ...) — TypeName.MethodName
Serialisation	Go's encoding/gob — only works between Go programs
Fields must be exported	Lowercase fields are silently ignored by gob — causes mysterious bugs
Method signature	Must be: func (h *T) Method(args *A, reply *R) error

Test net/rpc After Tasks 1-4

```
# Recompile server and client first
# (inside lab04-server) go build -o server_bin . && pkill server_bin &&
./server_bin &
# (inside lab04-client) go build -o client_bin .
```

```
# Test
docker exec lab04-client /lab04/client/client_bin rpc put city London
# Expected: [net/rpc] Stored key="city" value="London"

docker exec lab04-client /lab04/client/client_bin rpc get city
# Expected: [net/rpc] key="city" value="London"

docker exec lab04-client /lab04/client/client_bin rpc list
# Expected: [net/rpc] 1 keys: [city]
```

Part 2 — gRPC (Tasks 5–8)

Task 5 is reading the generated files (no coding). Tasks 6, 7, 8 implement the gRPC server and client.

```
docker exec -it lab04-server bash
cd /lab04/server
cat proto/kvstore_grpc.pb.go   # Task 5 — read this
nano grpc_server.go             # Task 6
nano main.go                    # Task 8

docker exec -it lab04-client bash
cd /lab04/client
nano grpc_client.go             # Task 7
```

gRPC Tasks

Task 5 — server/proto/
`Read generated files` [Easy]
Read kvstore_grpc.pb.go — find the KeyValueStoreServer interface and understand what you must implement

Task 6 — server/grpc_server.go
`Put, Get, Delete, List` [Medium]
Implement all 4 methods on GRPCServer — call store methods, return pb.XxxResponse structs

Task 7 — client/grpc_client.go
`Connect, Put, Get, Delete, List, Close` [Medium]
grpc.Dial to connect, create pb.NewKeyValueStoreClient(conn), call methods with context

Task 8 — server/main.go
`startGRPCServer()` [Easy]
net.Listen on port 7001, grpc.NewServer(), pb.RegisterKeyValueStoreServer(), s.Serve(ln)

gRPC vs net/rpc — Side by Side

	net/rpc	gRPC
Service definition	Go structs (types.go)	kvstore.proto
Code generation	None — write by hand	protoc generates stubs
Server interface	Your own structs	Generated interface (must match exactly)
Client call	client.Call("KVHandler.Put", args, &reply)	client.Put(ctx, &pb.PutRequest{Key: key})
Context support	No	Yes — deadlines, cancellation
Language support	Go only	Any language
Protocol	Custom TCP + gob	HTTP/2 + Protocol Buffers

Test gRPC After Tasks 5-8

```
docker exec lab04-client /lab04/client/client_bin grpc put city London
# Expected: [gRPC] Stored key="city" value="London"

docker exec lab04-client /lab04/client/client_bin grpc get city
# Expected: [gRPC] key="city" value="London"

# Cross-protocol test — put via net/rpc, get via gRPC
docker exec lab04-client /lab04/client/client_bin rpc  put country UK
docker exec lab04-client /lab04/client/client_bin grpc get country
# Expected: key="country" value="UK"
# Both reach the same underlying store!
```

Part 3 — REST HTTP (Tasks 9–13)

```
docker exec -it lab04-server bash
cd /lab04/server
nano rest_server.go  # Tasks 9-12

docker exec -it lab04-client bash
cd /lab04/client
nano rest_client.go  # Task 13
```

REST API Design

HTTP Method	URL	Body	Response	Task
PUT	/keys/{key}	{ "value": "..." }	201 Created + {success: true}	10
GET	/keys/{key}	(none)	200 OK + {value:..., found:true} or 404	11
GET	/keys	(none)	200 OK + {keys:[...], count:N}	11
DELETE	/keys/{key}	(none)	200 OK + {deleted:true} or 404	12

REST Tasks

Task 9 — server/rest_server.go
`SetupRoutes()` [Easy]
Register /keys/ → handleKey and /keys → handleList using mux.HandleFunc

Task 10 — server/rest_server.go
`handlePut()` [Easy]
Decode JSON body, call store.Put(), respond 201 Created

Task 11 — server/rest_server.go
`handleGet() + handleList()` [Easy]
Get one key (200 or 404) and list all keys (200 with count)

Task 12 — server/rest_server.go
`handleDelete()` [Easy]
Delete key, respond 200 if deleted or 404 if not found

Task 13 — client/rest_client.go
`Put, Get, Delete, List` [Medium]
Use net/http to make HTTP requests, decode JSON responses

Test REST — Two Ways

Method 1 — from the client container:

```
docker exec lab04-client /lab04/client/client_bin rest put city London
```

```
docker exec lab04-client /lab04/client/client_bin rest get city
docker exec lab04-client /lab04/client/client_bin rest list
```

Method 2 — with curl from your LAPTOP terminal (REST only — not possible with net/rpc or gRPC):

```
curl -X PUT http://localhost:8080/keys/city \
     -H 'Content-Type: application/json' \
     -d '{"value":"London"}'

curl http://localhost:8080/keys/city

curl http://localhost:8080/keys

curl -X DELETE http://localhost:8080/keys/city
```

Why curl Works for REST but Not gRPC or net/rpc

REST uses standard HTTP — any HTTP client (browser, curl, Postman, Python requests) can call it. gRPC uses HTTP/2 with binary Protocol Buffers — needs a gRPC-aware client. net/rpc uses a custom Go-only protocol — only Go programs can call it. This is the key trade-off: REST is more accessible, gRPC is faster and more structured.

Part 4 — Benchmark and Compare

> ⚠ **All 13 Tasks Must Be Complete**
> Complete and test all 13 tasks before running the benchmark. If any protocol is not working, its benchmark result will not appear.

Run the benchmark from inside the client container:

```
docker exec -it lab04-client bash
cd /lab04/client
./client_bin bench 100

# Expected output:
# Protocol     Total time    Avg/op        Ops/sec
# net/rpc      45ms          225us         4444
# gRPC          38ms          190us         5263
# REST          210ms         1050us        952
# (your numbers will differ — what matters is the relative order)
```

Also run the all-protocols test to confirm cross-protocol data sharing:

```
# Put via one protocol, get via another
./client_bin rpc  put benchmark test123
./client_bin grpc get benchmark
./client_bin rest get benchmark
# All three should return: test123
```

Experiments

Experiment A — Cross-Protocol Data Sharing

1. Put 3 different keys using 3 different protocols (one key per protocol)
2. Read all 3 keys using all 3 protocols (9 reads total)
3. Record which combination worked and which failed

Observe and record:

* Can you read a key via gRPC that was written via net/rpc?
* Can you read a key via REST that was written via gRPC?
* What does this tell you about the relationship between the protocols and the store?

Experiment B — Benchmark: 100 Operations

1. Run: ./client_bin bench 100
2. Record the total time, average latency and ops/sec for each protocol
3. Run bench 500 and record again

Observe and record:

* Which protocol is fastest? By how much?
* Which is slowest? Why do you think REST is slower?
* Does the ranking change between 100 and 500 operations?

Experiment C — REST from Browser and curl

1. Open a web browser on your laptop
2. Go to: http://localhost:8080/keys — you should see a JSON list
3. Go to: http://localhost:8080/keys/city — you should see the value
4. Try calling the net/rpc server directly — is it possible?

Observe and record:

* Did the browser successfully call the REST server?
* Why can a browser call REST but not net/rpc or gRPC?
* What does this mean for building a web application that needs a backend?

Experiment D — Large Values

1. Create a large string value (~10,000 characters) and store it via each protocol

```
python3 -c "print('x'*10000)" > /tmp/big.txt
VAL=$(cat /tmp/big.txt)
./client_bin rpc  put bigkey $VAL
./client_bin grpc put bigkey $VAL
./client_bin rest put bigkey $VAL
```

2. Time how long each get takes: time ./client_bin rpc get bigkey

Observe and record:

* Which protocol handles large values fastest?
* Does the relative performance order change with large values?

Discussion Questions

Answer all four on Moodle. You may write notes here first.

1. You implemented Put, Get, Delete, List three times — once for net/rpc, once for gRPC, once for REST. Which implementation was easiest to write and which was hardest? Refer to specific code differences between the three.

Answer:

2. In Experiment C, a browser could call the REST server but not net/rpc or gRPC. Explain why. If you were building a web application that needed a backend API, which of the three protocols would you choose and why?

Answer:

3. net/rpc requires all fields to be exported (capital letter). A lowercase field compiles and runs fine but the value is always zero or empty on the server side. Explain WHY this happens — what is gob encoding doing, and what is the lesson for distributed systems design?

Answer:

4. In Labs 02 and 03 you used callRPC() which called net/rpc under the hood. Now that you have used gRPC as well, if you were rebuilding the Lab 02 Chord DHT today for a production system where clients might be written in Python or Java, which communication protocol would you choose? Justify your answer using what you observed in this lab.

Answer:

Results Recording

Experiment A — Cross-Protocol Reads

Written with →	Read via net/rpc	Read via gRPC	Read via REST
net/rpc	(same)		
gRPC		(same)	
REST			(same)

Experiment B — Benchmark Results

Protocol	100 ops total time	100 ops avg/op	100 ops/sec	500 ops avg/op
net/rpc				
gRPC				
REST				

Protocol Comparison

Property	net/rpc	gRPC	REST
Language support			
Fastest for high-throughput			
Works from a browser			
Needs code generation			
Best use case			

Submission

Item	Where	Format
Discussion questions (Q1-Q4)	Moodle — Lab 04 Quiz	Type answers in Moodle
Results tables	Moodle — Lab 04 Quiz	Fill in Moodle form

Troubleshooting

Problem	Fix
Build fails: go.mod not found	Run docker build from inside lab04-complete folder, not a subfolder
Build fails: cannot find module	Check internet connection — Docker needs to download gRPC packages
Server not starting after recompile	Check: go build printed any errors. Fix errors then rebuild
[net/rpc] Not implemented yet	Complete Task 3 — startRPCServer() in main.go
[gRPC] Not implemented yet	Complete Task 8 — startGRPCServer() in main.go
get returns empty string (not error)	Lowercase field in types.go — check Task 1, all fields must be Capital
grpc: connection refused on port 7001	gRPC server not started — Task 8 not complete
curl: connection refused on port 8080	REST server not started or Task 9 SetupRoutes not complete
gRPC: method not implemented	Tasks 6 methods still returning default — implement the TODO functions
Benchmark shows 0 for a protocol	That protocol is not working — fix it before benchmarking
Container stopped unexpectedly	docker logs lab04-server to see the error

Go Quick Reference

net/rpc

```
// Server — register and start
rpc.Register(&KVHandler{store: store})
ln, _ := net.Listen("tcp", ":7000")
rpc.Accept(ln)  // blocks — run in goroutine

// Client — connect and call
client, err := rpc.Dial("tcp", "lab04-server:7000")
var reply PutReply
err = client.Call("KVHandler.Put", &PutArgs{Key: "k", Value: "v"}, &reply)
client.Close()
```

gRPC

```
// Server — register generated interface
ln, _ := net.Listen("tcp", ":7001")
s := grpc.NewServer()
pb.RegisterKeyValueStoreServer(s, &GRPCServer{store: store})
s.Serve(ln)  // blocks — run in goroutine

// Server method signature (must match generated interface)
func (s *GRPCServer) Put(ctx context.Context, req *pb.PutRequest)
(*pb.PutResponse, error) {
    s.store.Put(req.Key, req.Value)
    return &pb.PutResponse{Success: true}, nil
}

// Client
conn, _ := grpc.Dial(addr,
grpc.WithTransportCredentials(insecure.NewCredentials()))
client := pb.NewKeyValueStoreClient(conn)
ctx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
defer cancel()
resp, err := client.Put(ctx, &pb.PutRequest{Key: "k", Value: "v"})
```

REST — HTTP Server

```
// Register routes
mux.HandleFunc("/keys/", s.handleKey)
mux.HandleFunc("/keys", s.handleList)
http.ListenAndServe(":8080", mux)

// Handler — decode JSON body
var body struct{ Value string `json:"value"` }
json.NewDecoder(r.Body).Decode(&body)

// Handler — send JSON response
w.Header().Set("Content-Type", "application/json")
w.WriteHeader(http.StatusCreated)  // 201
json.NewEncoder(w).Encode(map[string]bool{"success": true})

// Extract key from URL path
key := strings.TrimPrefix(r.URL.Path, "/keys/")

// Check HTTP method
switch r.Method {
case http.MethodPut:    // handle put
case http.MethodGet:    // handle get
```

```
case http.MethodDelete: // handle delete
}
```

REST — HTTP Client

```
// PUT request with JSON body
body, _ := json.Marshal(map[string]string{"value": value})
req, _ := http.NewRequest(http.MethodPut, url, bytes.NewBuffer(body))
req.Header.Set("Content-Type", "application/json")
resp, err := http.DefaultClient.Do(req)

// GET request
resp, err := http.Get(url)
defer resp.Body.Close()
body, _ := io.ReadAll(resp.Body)
var result map[string]interface{}
json.Unmarshal(body, &result)

// Check status code
if resp.StatusCode == http.StatusNotFound { /* key not found */ }
```